

EXPT NO. : 1 A STUDY OF BASIC DIGITAL ICS

DATE:

AIM:

To verify the truth table of basic digital ICs of AND, OR, NOT, NAND, NOR, EX-OR gates.

APPARATUS REQUIRED:

Sl.No.	COMPONENT	SPECIFICATION	QTY.
1	AND GATE	IC 7408	1
2	OR GATE	IC 7432	1
3	NOT GATE	IC 7404	1
4	NAND GATE	IC 7400	1
5	NOR GATE	IC 7402	1
6	X-OR GATE	IC 7486	1
7	IC TRAINER KIT	-	1
8	PATCH CORDS & WIRES	-	As needed

THEORY:

Logic gates are the basic elements that make up a digital system. The electronic gate is a circuit that is able to operate on a number of binary inputs in order to perform a particular logic function. The type of gates available are the NOT, AND, OR, NAND, NOR, Exclusive-OR and the Exclusive-NOR.

1. AND gate:

An AND gate is the physical realization of logical multiplication operation. It is an electronic circuit which generates an output signal of '1' only if all the input signals are '1'.

2. OR gate:

An OR gate is the physical realization of the logical addition operation. It is an electronic circuit which generates an output signal of '1' if any of the input signal is '1'.

3. NOT gate:

A NOT gate is the physical realization of the complementation operation. It is an electronic circuit which generates an output signal which is the reverse of the input signal. A NOT gate is also known as an inverter because it inverts the input.

4. NAND gate:

A NAND gate is a complemented AND gate. The output of the NAND gate will be '0' if all the input signals are '1' and will be '1' if any one of the input signal is '0'.

5.NOR gate:

A NOR gate is a complemented OR gate. The output of the OR gate will be '1' if all the inputs are '0' and will be '0' if any one of the input signal is '1'.

6.EX-OR gate:

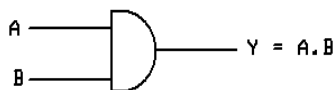
An Ex-OR gate performs the following Boolean function,

$$A \oplus B = (A \cdot B') + (A' \cdot B)$$

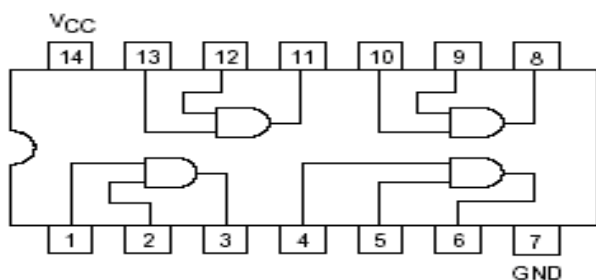
It is similar to OR gate but excludes the combination of both A and B being equal to one. The exclusive OR is a function that give an output signal '0' when the two input signals are equal either '0' or '1'

AND GATE

LOGIC DIAGRAM:



PIN DIAGRAM OF IC 7408:



TRUTH TABLE

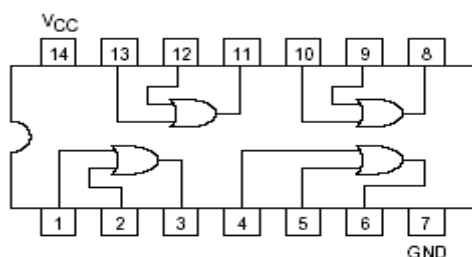
S.No	INPUT		OUTPUT
	A	B	$Y = A \cdot B$
1.	0	0	0
2.	0	1	0
3.	1	0	0
4.	1	1	1

OR GATE:

LOGIC DIAGRAM:



PIN DIAGRAM OF IC 7432 :

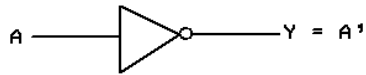


TRUTH TABLE

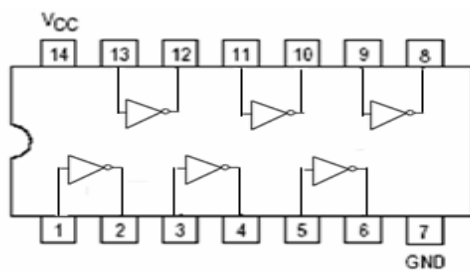
S.No	INPUT		OUTPUT
	A	B	$Y = A + B$
1.	0	0	0
2.	0	1	1
3.	1	0	1
4.	1	1	1

NOT GATE

LOGIC DIAGRAM:



PIN DIAGRAM OF IC 7404 :

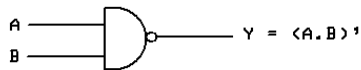


TRUTH TABLE:

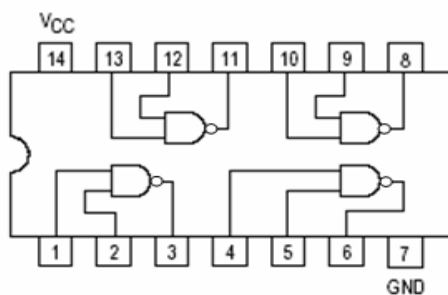
S.No	INPUT	OUTPUT
	A	$Y = A'$
1.	0	1
2.	1	0

NAND GATE

LOGIC DIAGRAM:



PIN DIAGRAM OF IC 7400:

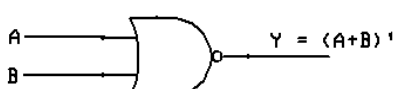


TRUTH TABLE

S.No	INPUT		OUTPUT
	A	B	$Y = (A . B)'$
1.	0	0	1
2.	0	1	1
3.	1	0	1
4.	1	1	0

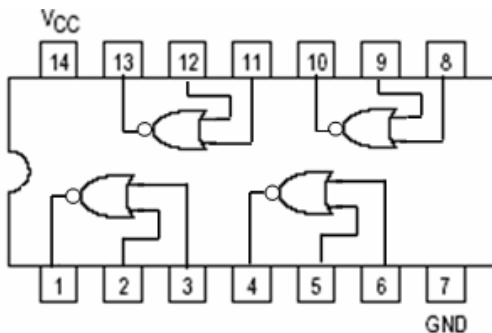
NOR GATE:

LOGIC DIAGRAM:



PIN DIAGRAM OF IC 7402 (NOR GATE)

TRUTH TABLE



S.No	INPUT		OUTPUT
	A	B	$Y = (A + B)'$
1.	0	0	1
2.	0	1	0
3.	1	0	0
4.	1	1	0

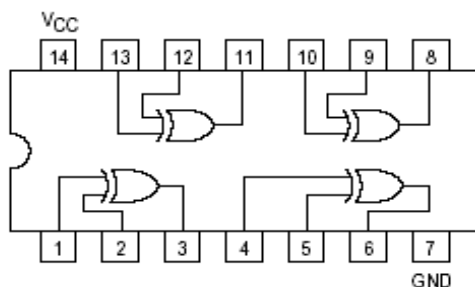
EX-OR GATE:

LOGIC DIAGRAM:



PIN DIAGRAM OF IC 7486 :

TRUTH TABLE



S.No	INPUT		OUTPUT
	A	B	$Y = A \oplus B$
1.	0	0	0
2.	0	1	1
3.	1	0	1
4.	1	1	0

PROCEDURE:

1. Connections are given as per the circuit diagram
2. For all the ICs 7th pin is grounded and 14th pin (V_{cc}) is given +5 V supply.
3. Apply the inputs and verify the truth table for all gates.

RESULT:

Thus the truth tables of digital ICs of AND, OR, NOT, NAND, NOR, EX-OR gates are verified.

EXP. NO : 1 B VERIFICATION OF BOOLEAN THEOREMS & LAWS**DATE :****Aim:**

To verify the truth tables of DeMorgan's theorem and Boolean algebraic laws by using logic gates.

Apparatus Required:

S.NO	COMPONENTS	SPECIFICATION	QUANTITY
1	Digital IC trainer kit	-	1
2	IC	7400	1
		7402	1
		7404	1
		7408	1
		7432	1
		7486	1
3	Patch Cords & Wires	-	As required

Theory:**DeMorgan's Theorems****First Theorem:**

It states that the complement of a product is equal to the sum of the complements.

$$(AB)' = A' + B'$$

Second Theorem:

It states that the complement of a sum is equal to the product of the complements.

$$(A+B)' = A' \cdot B'$$

Boolean Laws:

Boolean algebra is a mathematical system consisting of a set of two or more distinct elements, two binary operators denoted by the symbols (+) and (.) and one unary operator denoted by the symbol either bar (-) or prime ('). They satisfy the commutative, associative, distributive and absorption properties of the Boolean algebra.

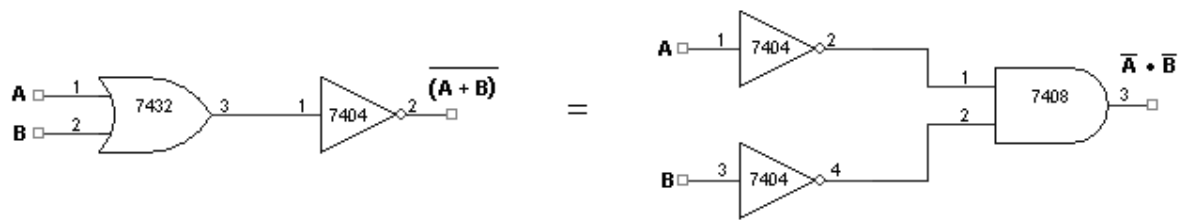
Commutative Property:

Boolean addition is commutative, given by

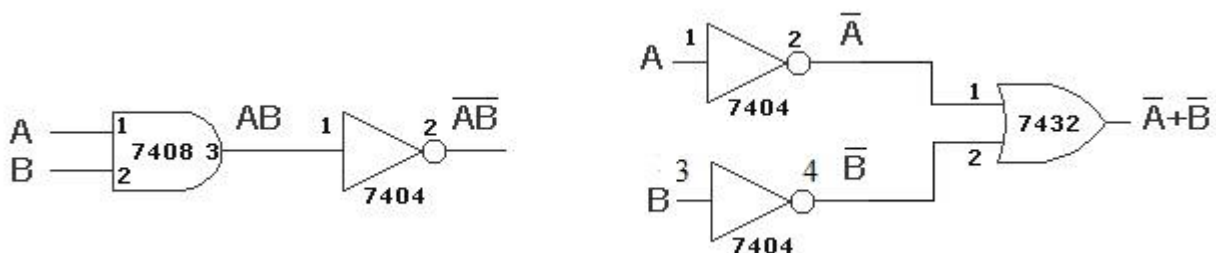
$$A+B=B+A$$

Boolean algebra is also commutative over multiplication, given by

$$A.B=B.A$$

De-Morgan's Theorem: 1**Truth Table**

Input		Output	
A	B	$(A+B)'$	$A' \cdot B'$
0	0	1	1
0	1	0	0
1	0	0	0
1	1	0	0

De-Morgan's Theorem: 2

Input		Output	
A	B	$(A.B)'$	$A' + B'$
0	0	1	1
0	1	1	1
1	0	1	1
1	1	0	0

Associative Property:

The associative property of addition is given by

$$A + (B+C) = (A+B) + C$$

The associative law of multiplication is given by $A \cdot (B.C) = (A.B).C$

Distributive Property:

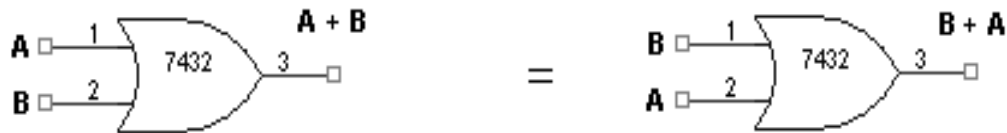
The Boolean addition is distributive over Boolean multiplication, given by

$$A+BC = (A+B) (A+C)$$

Boolean multiplication is also distributive over Boolean addition given by

$$A \cdot (B+C) = A \cdot B + A \cdot C$$

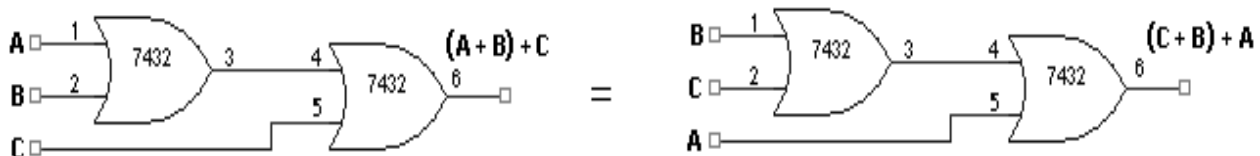
Commutative Law:



Truth Table:

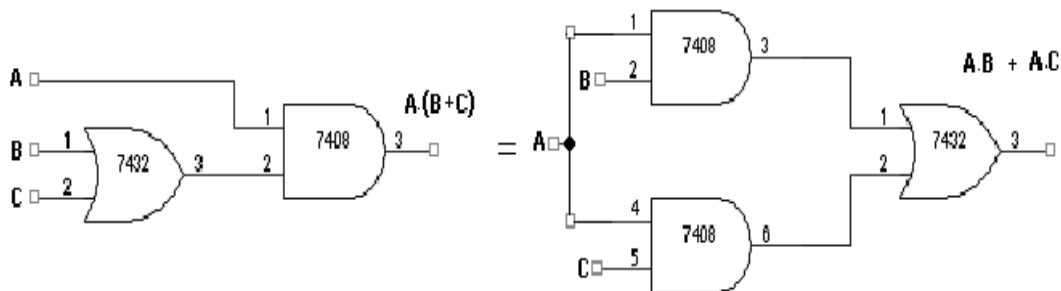
Input		Output	
A	B	A+B	B+A
0	0	0	0
0	1	1	1
1	0	1	1
1	1	1	1

Associative Law:



Truth table:

Input			Output			
A	B	C	A+B	(A+B)+C	B+C	A+(B+C)
0	0	0	0	0	0	0
0	0	1	0	1	1	1
0	1	0	1	1	1	1
0	1	1	1	1	1	1
1	0	0	1	1	0	1
1	0	1	1	1	1	1
1	1	0	1	1	1	1
1	1	1	1	1	1	1

Distributive Law:**Truth Table:**

Input			Output				
A	B	C	B+C	A.(B+C)	A.B	A.C	A.B+A.C
0	0	0	0	0	0	0	0
0	0	1	1	0	0	0	0
0	1	0	1	0	0	0	0
0	1	1	1	0	0	0	0
1	0	0	0	0	0	0	0
1	0	1	1	1	0	1	1
1	1	0	1	1	1	0	1
1	1	1	1	1	1	1	1

Procedure:

1. Connections are made as per the circuit/logic diagram.
2. Make sure that the ICs are enabled by giving the suitable V_{cc} and ground connections.
3. Apply the logic inputs and observe the output by verifying the truth table given.

Result :

Thus DeMorgan's theorem and Boolean laws are verified using truth tables.

Ex.No:2 a.

IMPLEMENTATION OF BOOLEAN FUNCTION

Date:

Aim :

To implement the given Boolean functions using Basic logic gates

Apparatus required :

Sl.No.	COMPONENT	SPECIFICATION	QTY.
1	3-INPUT AND GATE	IC7411	1
2	AND GATE	IC 7408	1
3	OR GATE	IC 7432	1
4	NOT GATE	IC 7404	1
5	IC TRAINER KIT	-	1
6	PATCH CORDS& Wires	-	As needed

Theory :

A Boolean function described by an algebraic expression consists of binary variables, the constants 0 & 1, and the logic operation symbols. When a Boolean expression is implemented with logic gates, each term requires a gate & each variable within the term designates an input to the gate. The Boolean functions are expressed either in sum of product (SOP) form for NAND implementation (or) product of sum (POS) form for NOR implementation. The given Boolean function is minimized using K map.

Boolean function implementation :**a) Realize $F(A, B, C, D) = \Pi(0, 1, 2, 5, 8, 9, 10)$ using Basic gates.****K – map reduction**

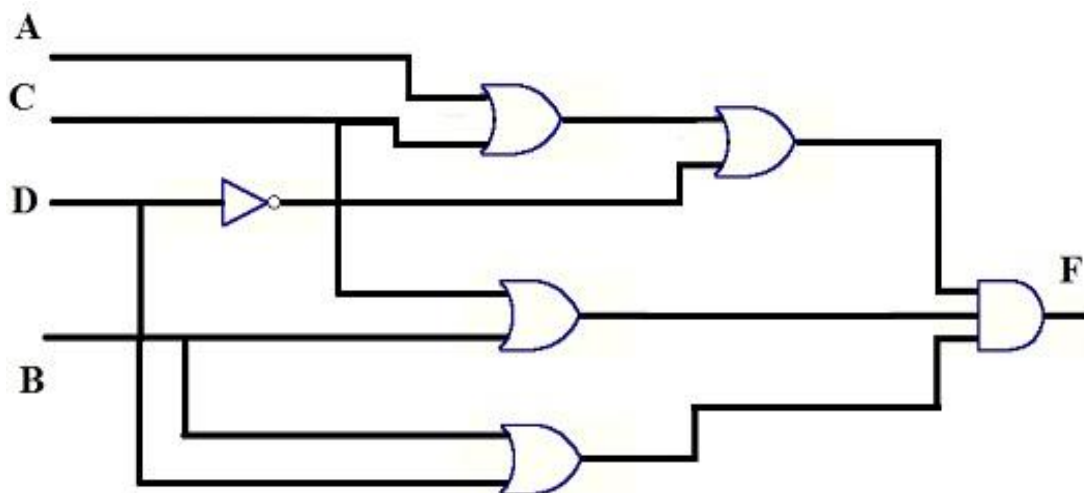
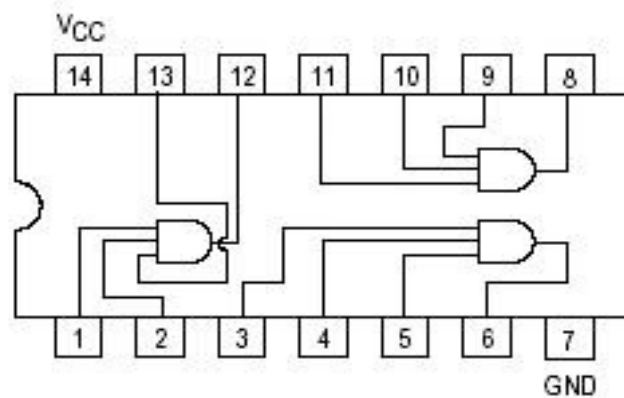
AB \ CD	CD			
	00	01	11	10
00	0	0		0
01		0		
11				
10	0	0		0

$$F = (A + C + D')(B + C)(B + D)$$

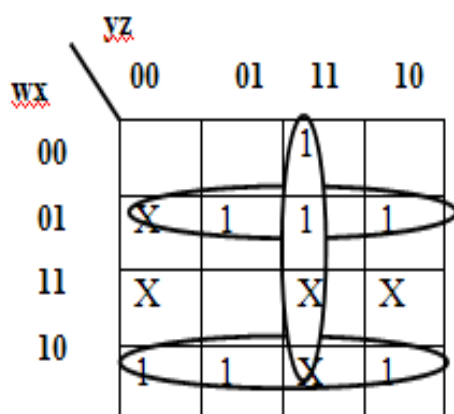
Truth table

A	B	C	D	F
0	0	0	0	0
0	0	0	1	0
0	0	1	0	0
0	0	1	1	1
0	1	0	0	1
0	1	0	1	0
0	1	1	0	1
0	1	1	1	1
1	0	0	0	0
1	0	0	1	0
1	0	1	0	0
1	0	1	1	1
1	1	0	0	1
1	1	0	1	1
1	1	1	0	1
1	1	1	1	1

IC7411(3 Input AND gate)

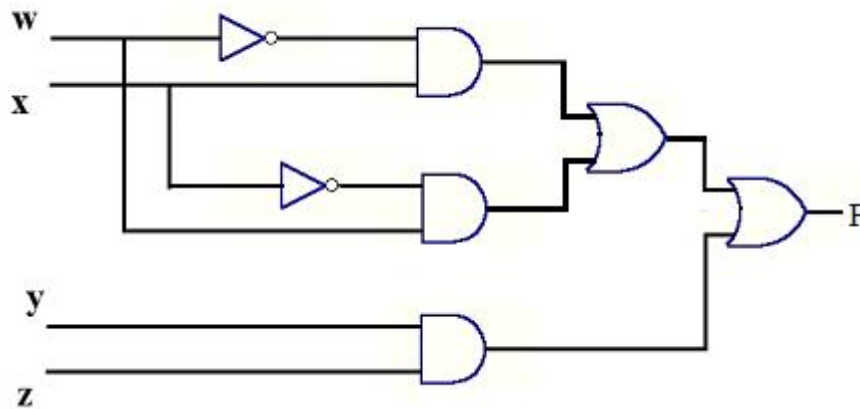


B] Realize $f(w, x, y, z) = \Sigma (3, 5, 6, 7, 8, 9, 10) + d(4, 11, 12, 14, 15)$ using logic gates.



$$f = \bar{w}x + w\bar{x} + yz$$

W	X	Y	Z	F
0	0	0	0	0
0	0	0	1	0
0	0	1	0	0
0	0	1	1	1
0	1	0	0	d
0	1	0	1	1
0	1	1	0	1
0	1	1	1	1
1	0	0	0	1
1	0	0	1	1
1	0	1	0	1
1	0	1	1	d
1	1	0	0	d
1	1	0	1	0
1	1	1	0	d
1	1	1	1	d



Procedure :

1. Connections are made as per the circuit/logic diagram.
2. Make sure that the ICs are enabled by giving the suitable V_{cc} and ground connections.
3. Apply the logic inputs and observe the output by verifying the truth table given.

Viva questions:

1. What is a combinational circuit?
2. What is minterm?
3. What is maxterm?
4. What is SOP & POS?
5. Why K-Map is required in Boolean algebra?

Result :

Thus the given Boolean function is designed using gates and verified with its truth table.

Ex.No: 2 B.

IMPLEMENTATION OF CODE CONVERTERS

DATE:

AIM:

To design, construct and test the following code converters.

1. Binary – Gray code converter (4 bit)
2. Gray - Binary code converter (4 bit)

APPARATUS REQUIRED:

Sl.No.	COMPONENT	SPECIFICATION	QTY.
1	X-OR GATE	IC 7486	1
2	IC TRAINER KIT	-	1
3	PATCH CORDS & WIRES	-	As needed

THEORY:

Code converter is a circuit that makes two systems compatible even though each uses a different code. In order that conversion from binary code to gray code & gray code to binary code, a combinational circuit code converter can be implemented with gates.

Binary code has two elements '0' & '1' each bit of binary code has a weight of 2. Gray code also has '0' & '1' as elements. But in this code there is no weight for bits.

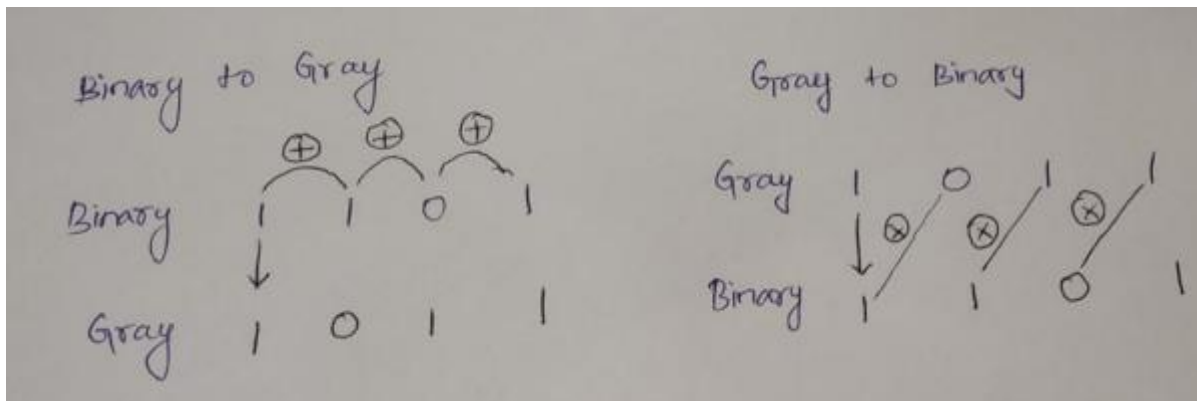
Binary to Gray conversion:

The most significant bit w is obtained directly from binary code. The second bit x is obtained by XORing a & b . Likewise all other bits are obtained in gray code by XORing the corresponding & preceding bits of binary code.

Gray to binary conversion:

The MSB is obtained directly from the MSB and the other bits are obtained by XORing the corresponding bit in gray code with preceding binary bit.

Example:



Binary to Gray code converter:

Binary code				Gray Code			
A	b	c	d	W	X	Y	Z
0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	1
0	0	1	0	0	0	1	1
0	0	1	1	0	0	1	0
0	1	0	0	0	1	1	0
0	1	0	1	0	1	1	1
0	1	1	0	0	1	0	1
0	1	1	1	0	1	0	0
1	0	0	0	1	1	0	0
1	0	0	1	1	1	0	1
1	0	1	0	1	1	1	1
1	0	1	1	1	1	1	0
1	1	0	0	1	0	1	0
1	1	0	1	1	0	1	1
1	1	1	0	1	0	0	1
1	1	1	1	1	0	0	0

Karnaugh map reduction:

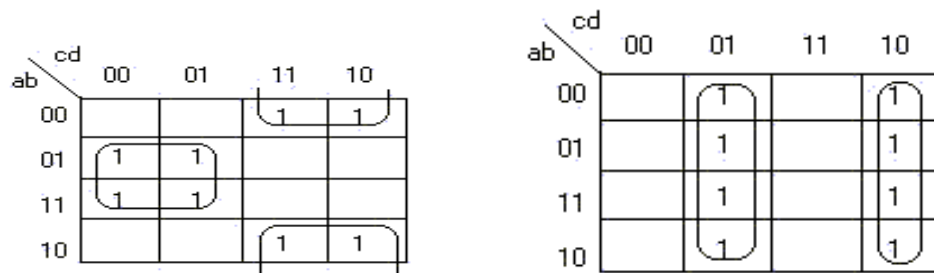
The K-maps for W, X, Y, Z based on the truth table & the minimized expressions from them are shown below.

cd \ ab	00	01	11	10
00				
01				
11	1	1	1	1
10	1	1	1	1

$$W=a$$

cd \ ab	00	01	11	10
00				
01	1	1	1	1
11				
10	1	1	1	1

$$X=a^1b + a b^1 = a \oplus b$$

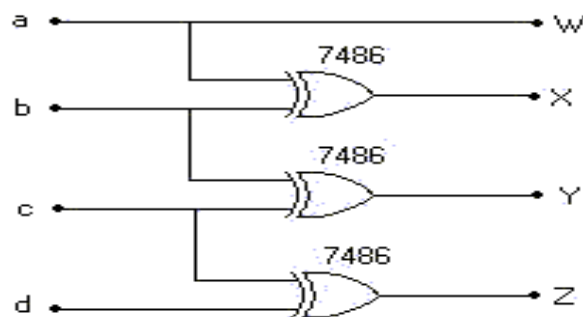


$$Y = bc^1 + b^1c = b \oplus c$$

$$Z = c^1d + cd^1 = c \oplus d$$

CIRCUIT DIAGRAM:

Binary to gray code converters



Gray to Binary Code Converter:

Gray code				Binary Code			
W	X	Y	Z	A	B	C	D
0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	1
0	0	1	1	0	0	1	0
0	0	1	0	0	0	1	1
0	1	1	0	0	1	0	0
0	1	1	1	0	1	0	1
0	1	0	1	0	1	1	0
0	1	0	0	0	1	1	1
1	1	0	0	1	0	0	0
1	1	0	1	1	0	0	1
1	1	1	1	1	0	1	0
1	1	1	0	1	0	1	1
1	0	1	0	1	1	0	0
1	0	1	1	1	1	0	1
1	0	0	1	1	1	1	0
1	0	0	0	1	1	1	1

The K-maps for A, B, C & D based on the truth table & the minimized expressions from them are shown below.

YZ \ WX	00	01	11	10
00				
01				
11	1	1	1	1
10	1	1	1	1

$$A=W$$

YZ \ WX	00	01	11	10
00				
01	1	1	1	1
11				
10	1	1	1	1

$$B=W^1X + WX^1 = W \oplus X$$

YZ \ WX	00	01	11	10
00			1	1
01	1	1		
11			1	1
10	1	1		

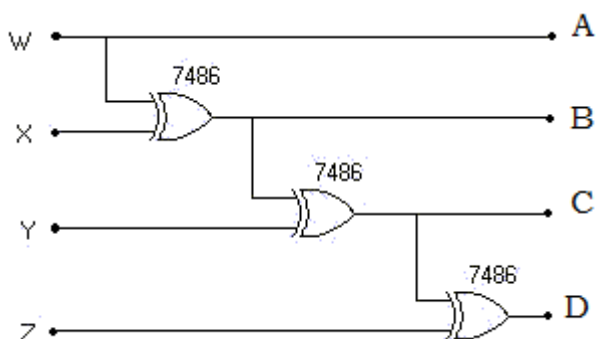
$$\begin{aligned} C &= W^1XY^1 + WX^1Y^1 + W^1X^1Y + WXY \\ &= (W^1X + WX^1)Y^1 + (W^1X^1 + WX)Y \\ &= W \oplus X \oplus Y \end{aligned}$$

CIRCUIT DIAGRAM

YZ \ WX	00	01	11	10
00		1		1
01	1		1	
11		1		1
10	1		1	

$$D = W \oplus X \oplus Y \oplus Z$$

Gray to binary code converter



Procedure:

1. Connections are made as per the circuit/logic diagram.
2. Make sure that the ICs are enabled by giving the suitable V_{cc} and ground connections.
3. Apply the logic inputs and observe the output by verifying the truth table given.

Result:

Thus Binary to Gray and Gray to Binary Code converters were designed and verified with the truth tables.

Ex.No: 2 C.

IMPLEMENTATION OF CODE CONVERTERS

DATE:

AIM:

To design, construct and test Excess 3 to BCD code converter

APPARATUS REQUIRED:

Sl.No.	COMPONENT	SPECIFICATION	QTY.
1	AND GATE	IC 7408	1
2	OR GATE	IC 7432	1
3	NOT GATE	IC 7404	1
4	X-OR GATE	IC 7486	1
5	IC TRAINER KIT	-	1
6	PATCH CORDS & WIRES	-	As needed

EXCESS-3 TO BCD CONVERTOR

TRUTH TABLE:

Excess – 3 Input				BCD Output			
X1	X2	X3	X4	A	B	C	D
0	0	1	1	0	0	0	0
0	1	0	0	0	0	0	1
0	1	0	1	0	0	1	0
0	1	1	0	0	0	1	1
0	1	1	1	0	1	0	0
1	0	0	0	0	1	0	1
1	0	0	1	0	1	1	0
1	0	1	0	0	1	1	1
1	0	1	1	1	0	0	0
1	1	0	0	1	0	0	1

K-Map for A:

X1 X2 \ X3 X4					
		00	01	11	10
00		x	x	0	x
01		0	0	0	0
11		1	x	x	x
10		0	0	1	0

$$A = X_1 X_2 + X_1 X_3 X_4$$

K-Map for B:

X3 X4 X1 X2		00	01	11	10
		00	01	11	10
00	X	X	0	X	
01	0	0	1	0	
11	0	X	X	X	
10	1	1	0	1	

$$B = [X_2 \oplus (X_3 \cdot X_4)]$$

K-Map for C:

X3 X2		X3 X4			
		00	01	11	10
X1 X2	00	X	X	0	X
	01	0	1	X	1
	11	0	X	X	X
	10	X	1	0	1

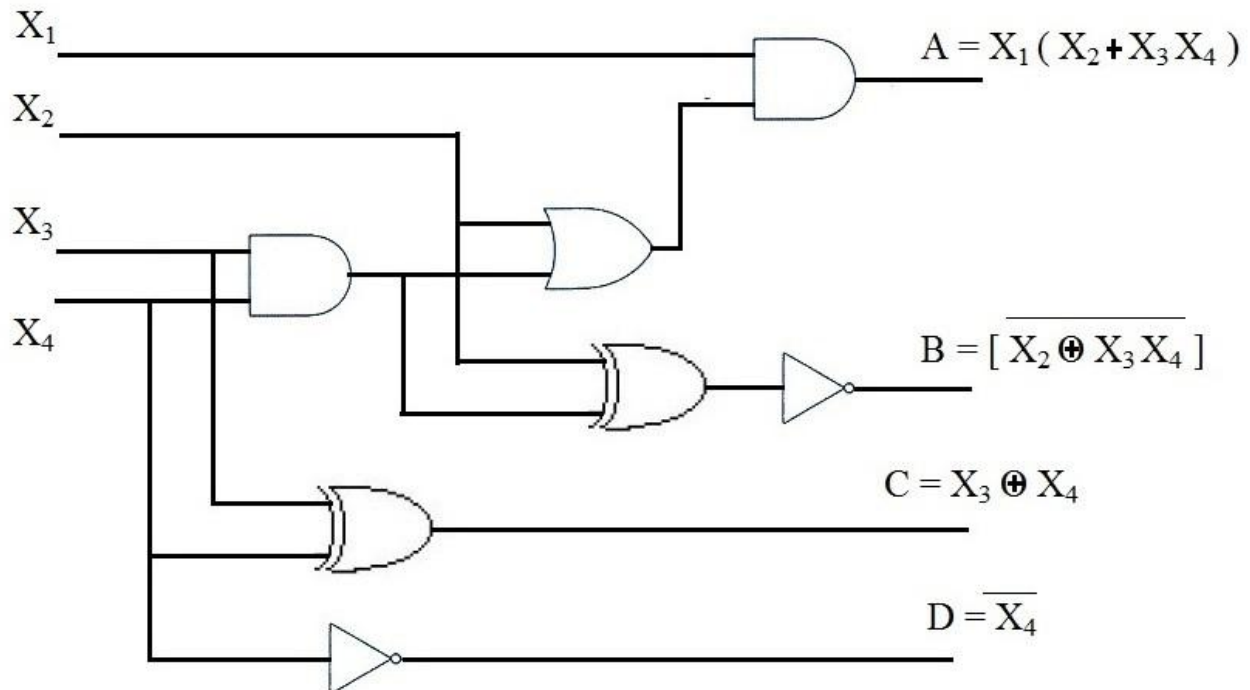
$$C = X_3 \oplus X_4$$

K-Map for D:

X1 X2 \ X3 X4		00	01	11	10
		00	01	11	10
00	X	X	0	X	
01	1	0	0	1	
11	1	X	X	X	
10	1	0	0	1	

$$D = \overline{X_4}$$

EXCESS-3 TO BCD CONVERTOR

**PROCEDURE:**

1. Connections are made as per the circuit/logic diagram.
2. Make sure that the ICs are enabled by giving the suitable V_{cc} and ground connections.
3. Apply the logic inputs and observe the output by verifying the truth table given.

Viva Questions :

1. What is unit distance code?
2. What are weighted & non weighted codes?
3. What is gray code?
4. What is BCD code?
5. What is self complementing code?

RESULT:

Thus Excess-3 to BCD code converter is designed using gates and verified with the truth table.

EXPT NO.: 3 DESIGN OF ADDER AND SUBTRACTOR**DATE :****AIM:**

To design and implement half adder, full adder, half subtractor and full subtractor circuits and verify the truth table using logic gates.

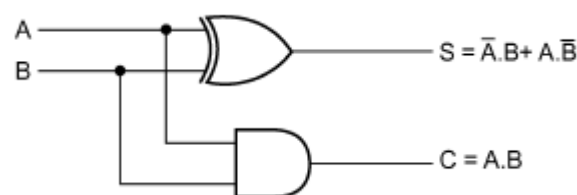
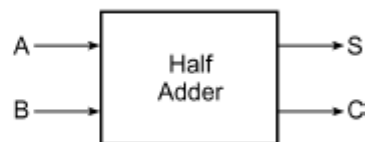
APPARATUS REQUIRED:

Sl.No.	COMPONENT	SPECIFICATION	QTY.
1.	AND GATE	IC 7408	1
2.	X-OR GATE	IC 7486	1
3.	NOT GATE	IC 7404	1
4.	OR GATE	IC 7432	1
5.	IC TRAINER KIT	-	1
6.	PATCH CORDS	-	As needed

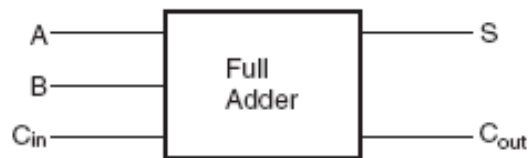
THEORY:**HALF ADDER:**

A *half-adder* is an arithmetic circuit block that can be used to add two bits. Such a circuit thus has two inputs that represent the two bits to be added and two outputs, with one producing the SUM output and the other producing the CARRY.

A	B	S	C
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

**FULL ADDER:**

A *full adder* circuit is an arithmetic circuit block that can be used to add three bits to produce a SUM and a CARRY output. Such a building block becomes a necessity when it comes to adding binary numbers with a large number of bits. The full adder circuit overcomes the limitation of the half-adder, which can be used to add two bits only.



A	B	C_{in}	SUM (S)	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

K-Map for SUM

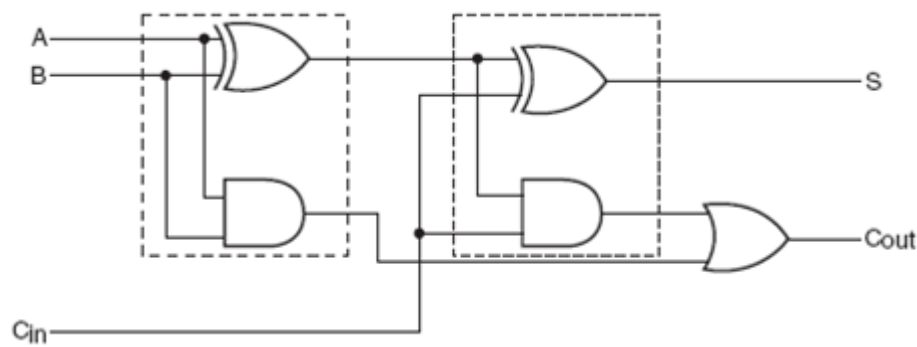
A \ BC_{in}	00	01	11	10
0		1		1
1	1		1	

$$SUM = A'B'C_{in} + A'BC'_{in} + ABC'_{in} + ABC_{in} = A \oplus B \oplus C_{in}$$

K-Map for CARRY

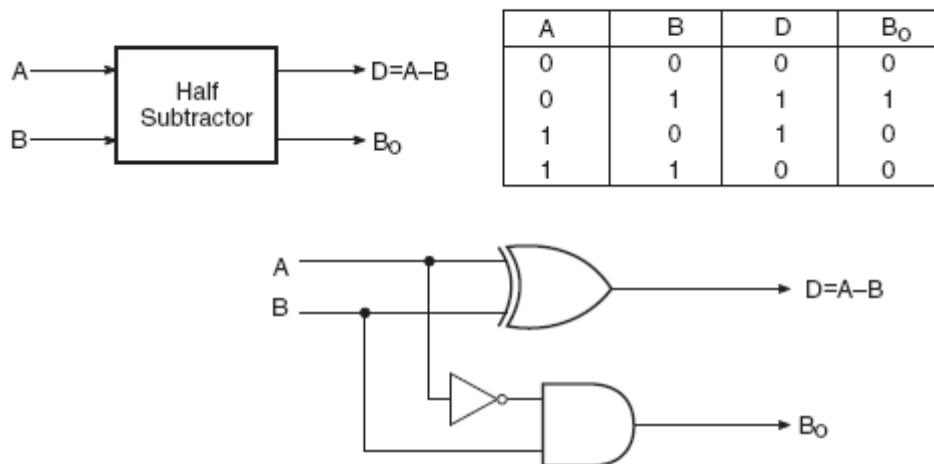
A \ BC_{in}	00	01	11	10
0			1	
1		1	1	1

$$CARRY = AB + BC_{in} + AC_{in}$$



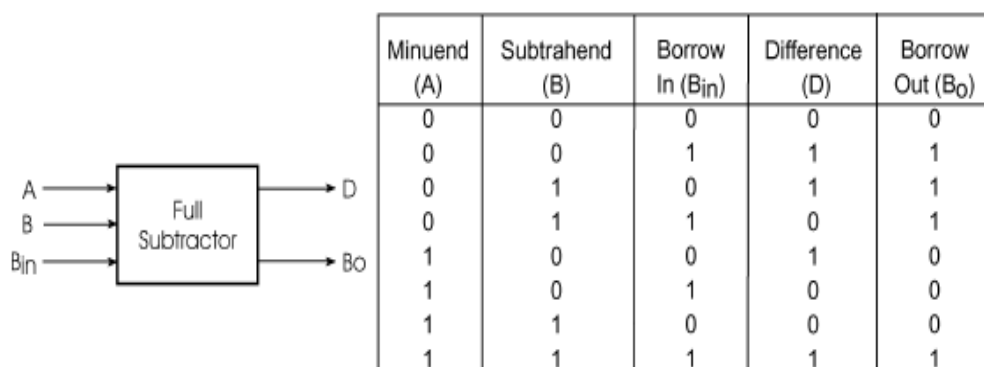
HALF SUBTRACTOR:

A *half-subtractor* is a combinational circuit that can be used to subtract one binary digit from another to produce a DIFFERENCE output and a BORROW output. The BORROW output here specifies whether a '1' has been borrowed to perform the subtraction.



FULL SUBTRACTOR:

There are three bits to be handled at the input of a full subtractor, namely the two bits to be subtracted and a borrow bit designated as Bin . There are two outputs, namely the DIFFERENCE output D and the BORROW output Bo.



K MAP for Difference (D)

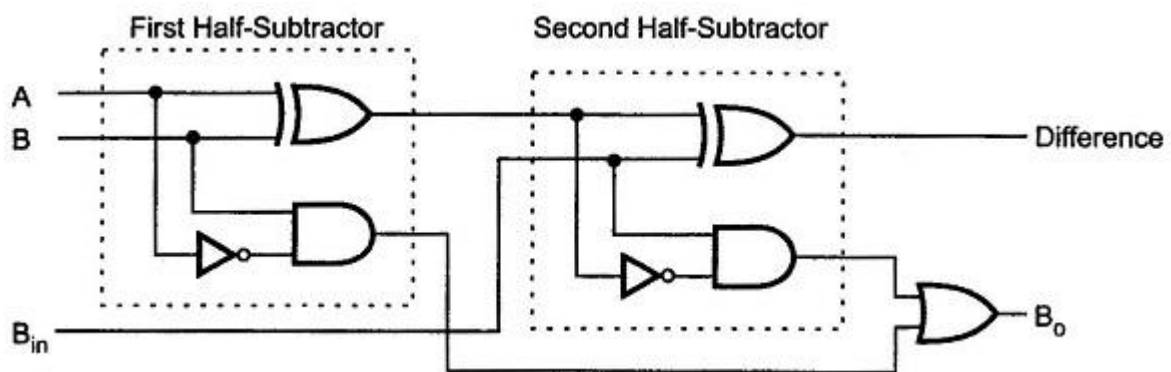
A \ B B _{in}	00	01	11	10
0		1		1
1	1		1	

$$\text{Difference } D = A \oplus B \oplus B_{in}$$

K MAP for Borrow out

A \ B B _{in}	00	01	11	10
0		1	1	1
1			1	

$$\text{Borrow out } B_0 = A'B_{in} + BB_{in} + A'B$$

**PROCEDURE:**

1. Connections are made as per the circuit/logic diagram.
2. Make sure that the ICs are enabled by giving the suitable V_{cc} and ground connections.
3. Apply the logic inputs and observe the output by verifying the truth table given.

Viva Questions:

1. What is half-adder?
2. Write the truth table of full adder?
3. How a full adder can be developed using half adder?
4. Write about half-subtractor and full-subtractor.
5. What is universal gate?

Result :

Thus, the half adder, full adder, half subtractor and full subtractor circuits are designed and verified the truth table using logic gates.

Ex.No : 4 IMPLEMENTATION OF PARITY GENERATOR / CHECKER**Date:****AIM:**

To design and implement Even and Odd Parity Generator/Checker using logic gates and MSI device.

APPARATUS REQUIRED:

S.NO	COMPONENTS	CONFIGURATION	QUANTITY
1	Digital IC Trainer kit	-	1
2	IC	7486	1
		7404	1
		74180	1
3	Connecting wires	-	As required
4	Patch cords	-	As required

Theory:**Parity generator:**

A parity bit is a scheme of detecting error during transmitting of binary information. A parity bit is an extra bit included with a binary message to make the number of 1's either odd or even.

Parity generators are used in digital transmission system for the errorless transmission of digital data. A parity bit is added to the data before the transmission and it will be checked for the correctness at the receiver end. There are two types of parity systems, even parity and odd parity. In the even parity system if the number of 1's in the data word is odd, a 1 will be added as a parity bit to the data to make total number of 1's even. If the number of 1's even, a 0 bit will be added. In the odd parity system if the number of 1's in the data word is odd, a 0 will be added to make the number of 1's odd. Otherwise, a 1 is added to make it odd. The circuit shown in the figure is used as a parity generator as well as a checker. ABCD is the 4-bit data word. Pi and Po are the parity input and parity output respectively.

The circuit that generates the parity bit in the transmitter is called a parity generator. The circuit that checks the parity in the receiver is called parity checker.

The working of the circuit can be concluded as follows.**Work as a Parity generator:**

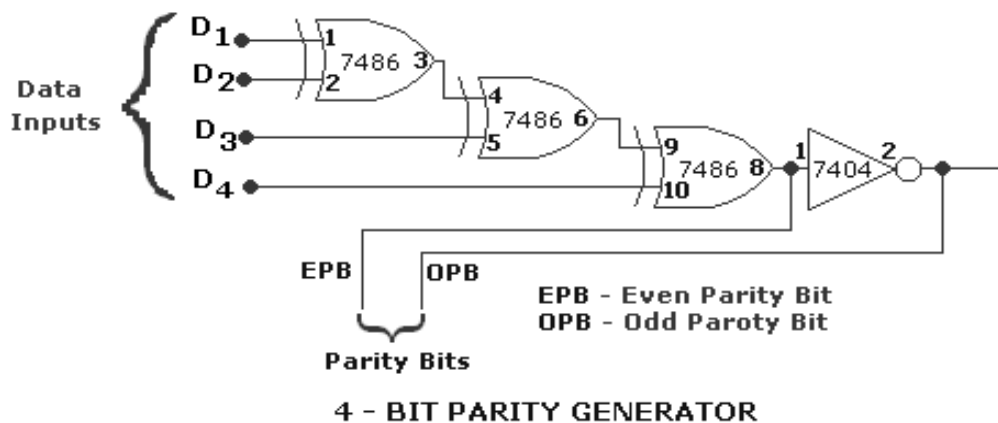
To generate an odd parity bit for ABCD, Pi must be made 0.

To generate an even parity bit for ABCD, Pi must be made 1.

Work as a parity checker:

If the parity of ABCD P_i is odd, P_o will be 0.

If the parity of ABCD P_i is even, P_o will be 1.

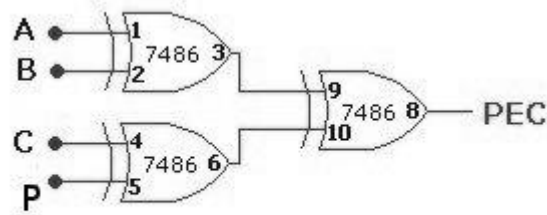
Circuit Diagram:

$$EPB = D_1 \oplus D_2 \oplus D_3 \oplus D_4$$

$$OPB = (D_1 \oplus D_2 \oplus D_3 \oplus D_4)'$$

Truth Table (Even and odd parity generator):

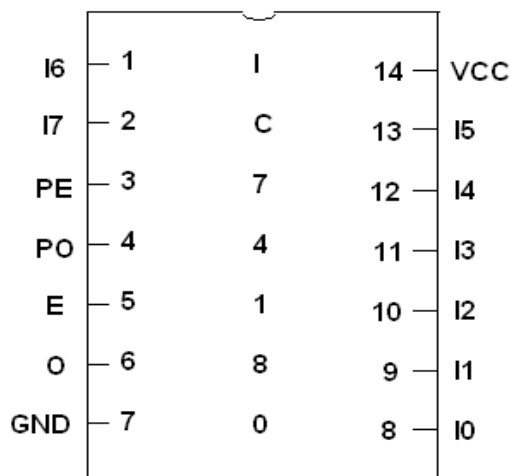
Data Inputs				Parity Bit	
D_1	D_2	D_3	D_4	EPB	OPB
0	0	0	0	0	1
0	0	0	1	1	0
0	0	1	0	1	0
0	0	1	1	0	1
0	1	0	0	1	0
0	1	0	1	0	1
0	1	1	0	0	1
0	1	1	1	1	0
1	0	0	0	1	0
1	0	0	1	0	1
1	0	1	0	0	1
1	0	1	1	1	0
1	1	0	0	0	1
1	1	0	1	1	0
1	1	1	0	1	0
1	1	1	1	0	1

Circuit Diagram for Even Parity Checker:

$$PEC = A \oplus B \oplus C \oplus P$$

4 - BIT PARITY CHECKER**Truth Table for Even Parity Checker**

4 – BIT DATA RECEIVED				PARITY ERROR CHECK
A	B	C	P	PEC
0	0	0	0	0
0	0	0	1	1
0	0	1	0	1
0	0	1	1	0
0	1	0	0	1
0	1	0	1	0
0	1	1	0	0
0	1	1	1	1
1	0	0	0	1
1	0	0	1	0
1	0	1	0	0
1	0	1	1	1
1	1	0	0	0
1	1	0	1	1
1	1	1	0	1
1	1	1	1	0

PIN DIAGRAM FOR IC 74180:**FUNCTION TABLE(Parity generator):**

INPUTS		OUTPUTS		
Number of High Data Inputs (I0 – I7)	PE	PO	ΣE	ΣO
EVEN	1	0	0	1
ODD	1	0	1	0
EVEN	0	1	1	0
ODD	0	1	0	1

PROCEDURE:

1. Connections are made as per the circuit/logic diagram.
2. Make sure that the ICs are enabled by giving the suitable V_{cc} and ground connections.
3. Apply the logic inputs and observe the output by verifying the truth table given.

Viva questions:

1. What is a parity bit?
2. Why parity is required?
3. What is parity generator?
4. What is parity checker?
5. What is an MSI device?

Result:

Thus Even and Odd Parity Generator and Checker using logic gates and MSI device are designed and verified with truth table.

EX NO.: 5 DESIGN AND IMPLEMENTATION OF MAGNITUDE COMPARATOR
DATE :

AIM:

To design and implement 4 – bit magnitude comparator using IC 7485.

APPARATUS REQUIRED:

Sl.No	COMPONENT	SPECIFICATION	QTY.
1	4-BIT MAGNITUDE COMPARATOR	IC 7485	1
2	IC TRAINER KIT	-	1
3	PATCH CORDS & WIRES	-	As needed

THEORY:

The comparison of two numbers is an operator that determine one number is greater than, less than (or) equal to the other number. A magnitude comparator is a combinational circuit that compares two numbers A and B and determine their relative magnitude. The outcome of the comparator is specified by three binary variables that indicate whether $A > B$, $A = B$ (or) $A < B$.

$$A = A_3 A_2 A_1 A_0$$

$$B = B_3 B_2 B_1 B_0$$

The equality of the two numbers and B is displayed in a combinational circuit designated by the symbol $(A=B)$.

This indicates A greater than B, then inspect the relative magnitude of pairs of significant digits starting from most significant position. A is 0 and that of B is 0.

We have $A < B$, the sequential comparison can be expanded as

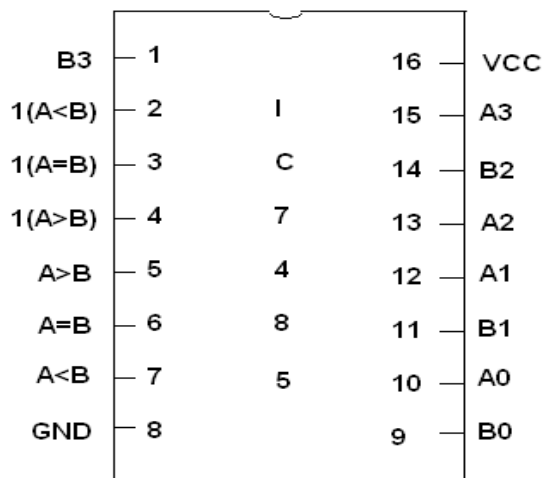
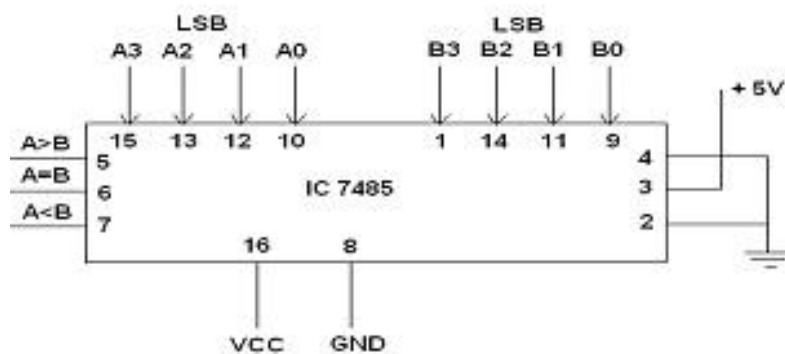
$$(A > B) = A_3 B_3^1 + X_3 A_2 B_2^1 + X_3 X_2 A_1 B_1^1 + X_3 X_2 X_1 A_0 B_0^1$$

$$(A < B) = A_3^1 B_3 + X_3 A_2^1 B_2 + X_3 X_2 A_1^1 B_1 + X_3 X_2 X_1 A_0^1 B_0$$

The same circuit can be used to compare the relative magnitude of two BCD digits.

Where, $A = B$ is expanded as,

$$(A = B) = (A_3 \oplus B_3)' (A_2 \oplus B_2)' (A_1 \oplus B_1)' (A_0 \oplus B_0)'$$

PIN DIAGRAM FOR IC 7485:**LOGIC DIAGRAM:****4 BIT MAGNITUDE COMPARATOR****TRUTH TABLE:****OUTPUT:**

A	B	A>B	A=B	A<B
0 0 0 0	0 0 0 0	0	1	0
0 0 0 1	0 0 0 0	1	0	0
0 0 0 0	0 0 0 1	0	0	1
1 0 0 0	0 0 0 1	1	0	0
0 1 0 1	1 0 0 0	0	0	1
0 1 0 1	0 1 0 1	0	1	0

PROCEDURE:

1. Connections are made as per the circuit/logic diagram.
2. Make sure that the ICs are enabled by giving the suitable V_{cc} and ground connections.
3. Apply the logic inputs and observe the output by verifying the truth table given.

Viva questions:

1. What is comparator?
2. Name the logic to be used to check inputs are same.
3. Name the logic to be used to check inputs are different.
4. Give an example for arithmetic circuits.
5. Give the importance of K-map in combinational circuits.

RESULT:

Thus the implementation of 4 – bit magnitude comparator using IC 7485 is verified with the truth tables.

EXPT NO. : 6 DESIGN AND IMPLEMENTATION OF MULTIPLEXER AND DEMULTIPLEXER**DATE :****AIM:**

To design and implement Multiplexer and Demultiplexer using logic gates.

APPARATUS REQUIRED:

Sl.No.	COMPONENT	SPECIFICATION	QTY.
1.	3 Input AND GATE	IC 7411	2
2.	OR GATE	IC 7432	1
3.	NOT GATE	IC 7404	1
4.	IC TRAINER KIT	-	1
5.	PATCH CORDS	-	As required

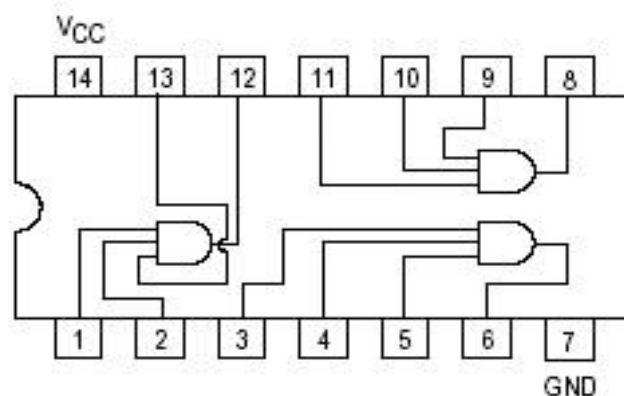
THEORY:**MULTIPLEXER:**

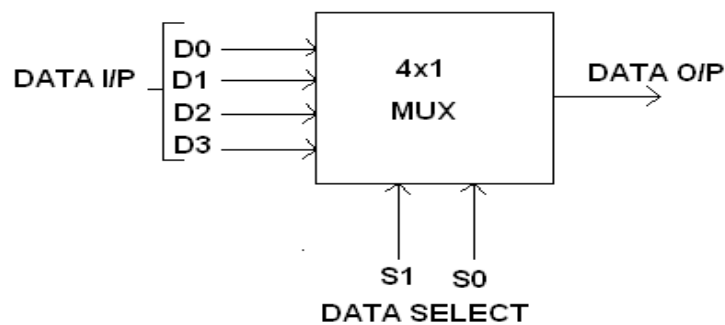
Multiplexer means transmitting a large number of information units over a smaller number of channels or lines. A digital multiplexer is a combinational circuit that selects binary information from one of many input lines and directs it to a single output line. The selection of a particular input line is controlled by a set of selection lines. Normally there are ' 2^n ' input line and ' n ' selection lines whose bit combination determine which input is selected.

DEMULTIPLEXER:

The function of Demultiplexer is in contrast to multiplexer function. It takes information from one line and distributes it to a given number of output lines. For this reason, the demultiplexer is also known as a data distributor. Decoder can also be used as demultiplexer.

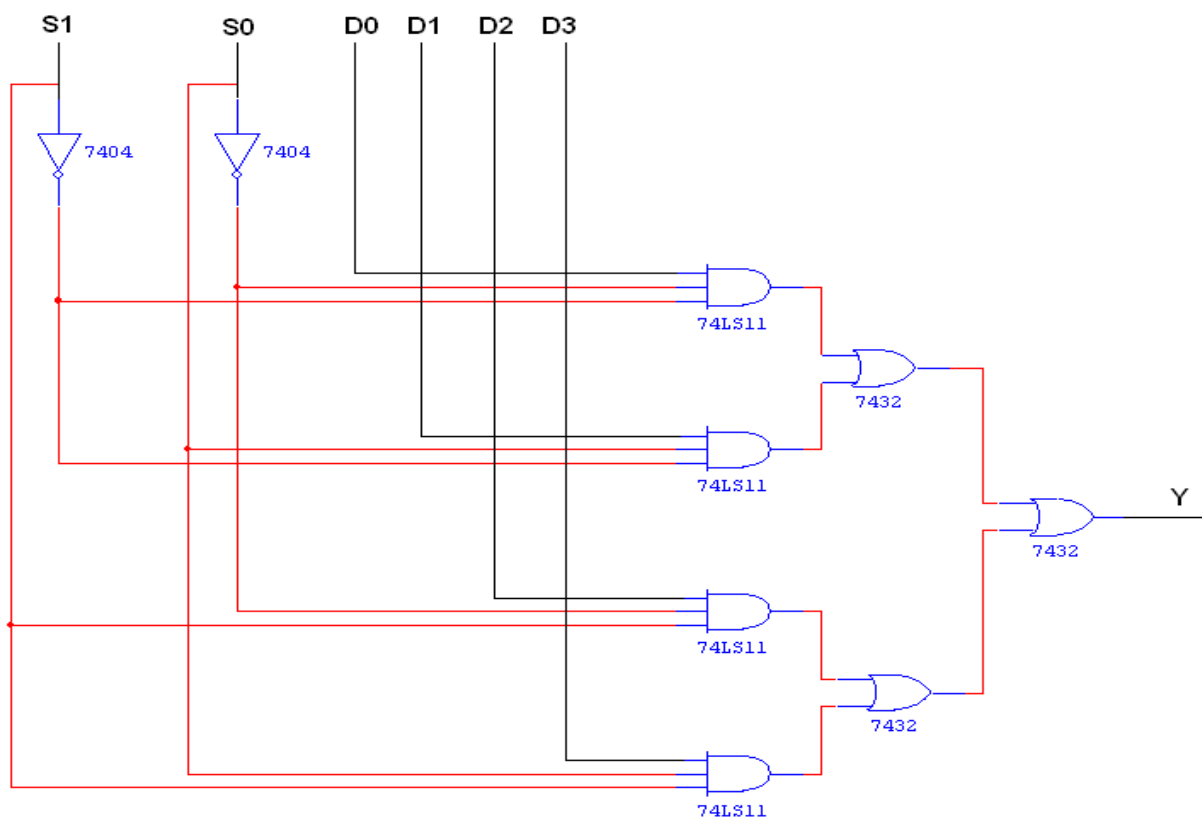
In the 1: 4 demultiplexer circuit, the data input line goes to all of the AND gates. The data select lines enable only one gate at a time and the data on the data input line will pass through the selected gate to the associated data output line.

PIN DIAGRAM OF IC7411(3 input AND gate)

BLOCK DIAGRAM FOR 4:1 MULTIPLEXER:**FUNCTION TABLE:**

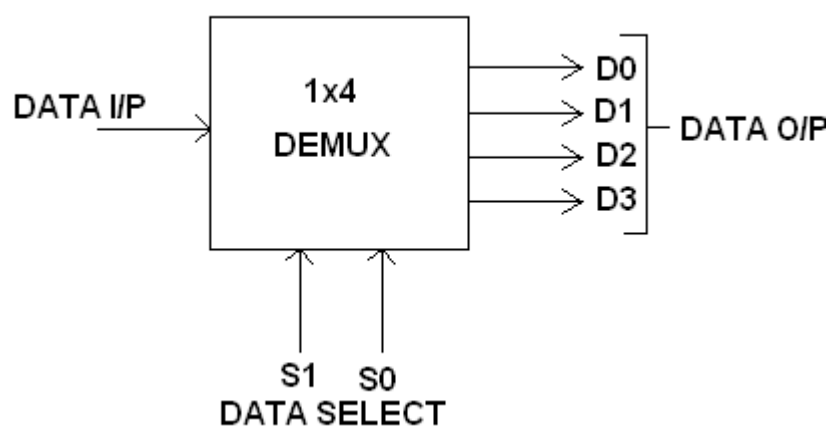
S1	S0	INPUTS Y
0	0	$D0 \rightarrow D0 S1' S0'$
0	1	$D1 \rightarrow D1 S1' S0$
1	0	$D2 \rightarrow D2 S1 S0'$
1	1	$D3 \rightarrow D3 S1 S0$

$$Y = D0 S1' S0' + D1 S1' S0 + D2 S1 S0' + D3 S1 S0$$

CIRCUIT DIAGRAM FOR MULTIPLEXER:

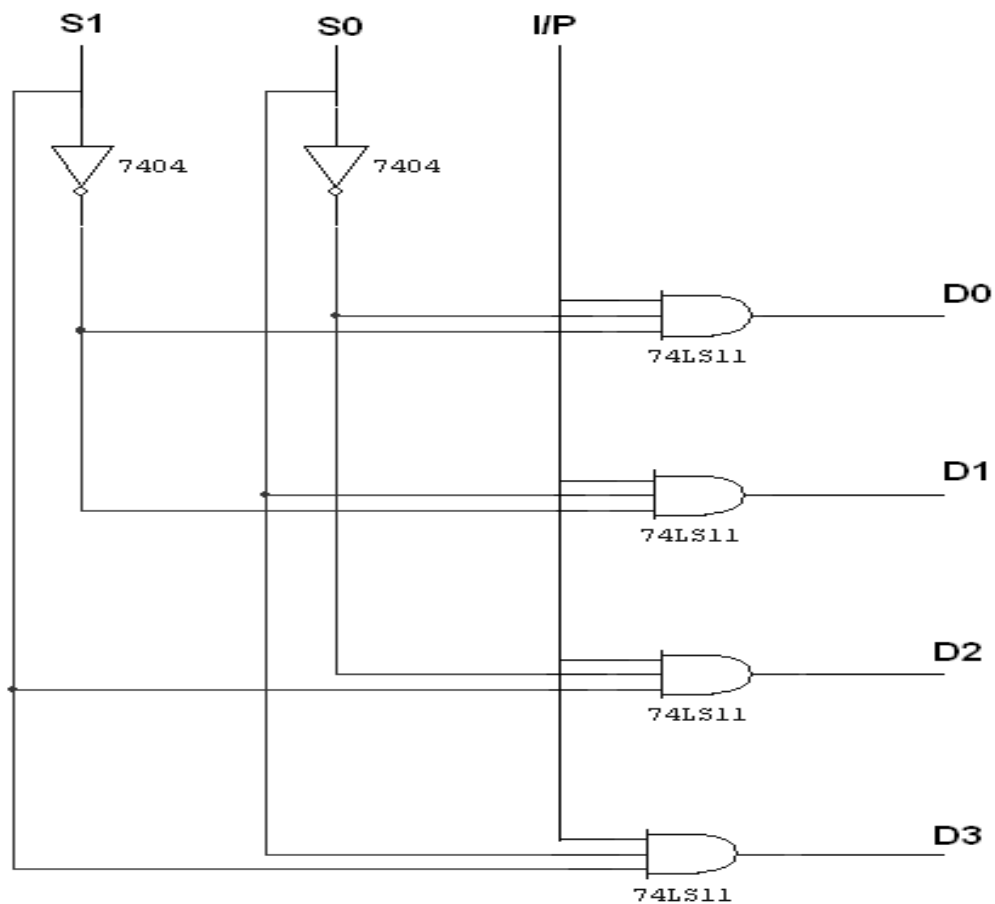
TRUTH TABLE of 1:4 DEMULTIPLEXER:

S1	S0	Y = OUTPUT
0	0	D0
0	1	D1
1	0	D2
1	1	D3

**FUNCTION TABLE:**

S1	S0	INPUT
0	0	$X \rightarrow D0 = X S1' S0'$
0	1	$X \rightarrow D1 = X S1' S0$
1	0	$X \rightarrow D2 = X S1 S0'$
1	1	$X \rightarrow D3 = X S1 S0$

$$Y = X S1' S0' + X S1' S0 + X S1 S0' + X S1 S0$$

LOGIC DIAGRAM FOR DEMULTIPLEXER:**TRUTH TABLE:**

INPUT			OUTPUT			
S1	S0	I/P	D0	D1	D2	D3
0	0	0	0	0	0	0
0	0	1	1	0	0	0
0	1	0	0	0	0	0
0	1	1	0	1	0	0
1	0	0	0	0	0	0
1	0	1	0	0	1	0
1	1	0	0	0	0	0
1	1	1	0	0	0	1

PROCEDURE:

1. Connections are made as per the circuit/logic diagram.
2. Make sure that the ICs are enabled by giving the suitable V_{cc} and ground connections.
3. Apply the logic inputs and observe the output by verifying the truth table given.

Viva questions:

1. What is a MUX?
2. What is the name for data selector?
3. What is DeMux?
4. Can you convert a Decoder into a DeMux? How?
5. What is known as data distributor? Why?

RESULT:

Thus a Multiplexer and Demultiplexer using logic gates are designed and the circuits are verified with the truth table.

EXPT NO.7 : CONSTRUCTION AND VERIFICATION OF 4 BIT RIPPLE COUNTER**DATE :****AIM:**

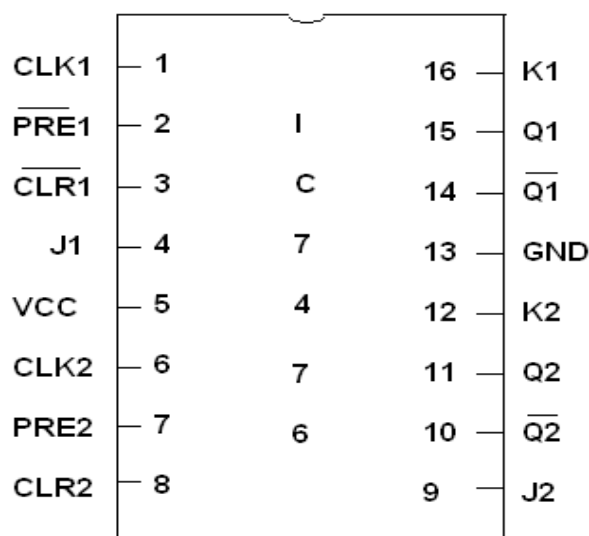
To design and verify 4 bit ripple counter.

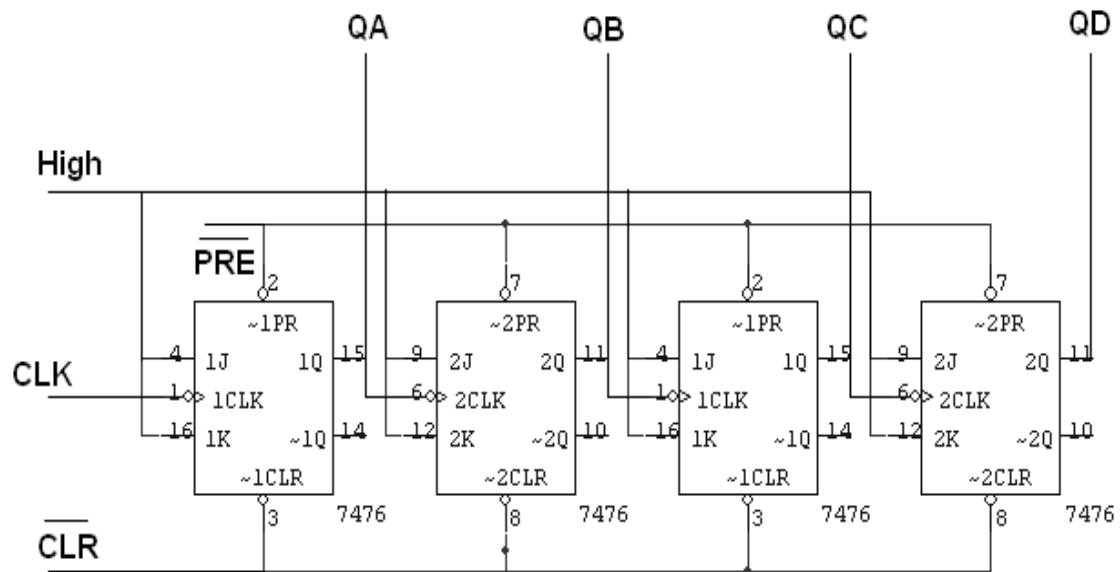
APPARATUS REQUIRED:

Sl.No.	COMPONENT	SPECIFICATION	QTY.
1.	JK FLIP FLOP	IC 7476	2
2	IC TRAINER KIT	-	1
3	PATCH CORDS	-	As required

THEORY:

A counter is a register capable of counting number of clock pulse arriving at its clock input. Counter represents the number of clock pulses arrived. A specified sequence of states appears as counter output. This is the main difference between a register and a counter. There are two types of counter, synchronous and asynchronous. In synchronous common clock is given to all flip flop and in asynchronous first flip flop is clocked by external pulse and then each successive flip flop is clocked by Q or $\bar{Q}$ output of previous stage. The clock of second stage is triggered by output of first stage. Because of inherent propagation delay time all flip flops are not activated at same time which results in asynchronous operation.

PIN DIAGRAM FOR IC 7476:

LOGIC DIAGRAM FOR 4 BIT RIPPLE COUNTER:**TRUTH TABLE:**

CLK	QA	QB	QC	QD
0	0	0	0	0
1	1	0	0	0
2	0	1	0	0
3	1	1	0	0
4	0	0	1	0
5	1	0	1	0
6	0	1	1	0
7	1	1	1	0
8	0	0	0	1
9	1	0	0	1
10	0	1	0	1
11	1	1	0	1
12	0	0	1	1
13	1	0	1	1
14	0	1	1	1
15	1	1	1	1

PROCEDURE:

1. Connections are made as per the circuit/logic diagram.
2. Make sure that the ICs are enabled by giving the suitable V_{cc} and ground connections.
3. Apply the logic inputs and observe the output by verifying the truth table given.

Viva Questions:

1. What is a sequential circuit?
2. What is Flip flop?
3. List the difference between Flipflop and latch.
4. What is synchronous circuit?
5. What is Asynchronous circuit?

RESULT:

Thus a 4 bit ripple counter using IC7476 is designed and verified with the truth table.

EXPT NO. 8 : DESIGN AND IMPLEMENTATION OF SHIFT REGISTER**DATE :****AIM:**

To design and implement the following in shift register

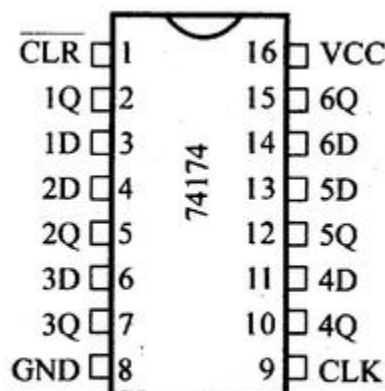
- Serial in serial out
- Serial in parallel out
- Parallel in serial out
- Parallel in parallel out

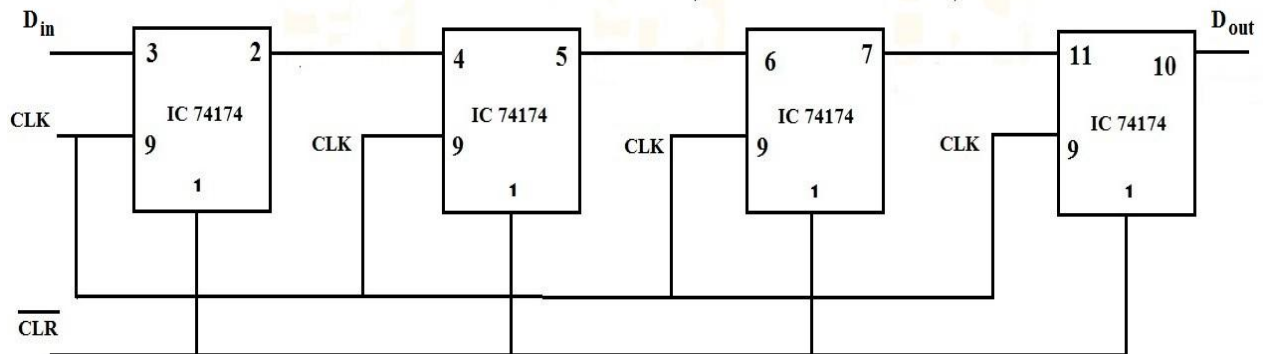
APPARATUS REQUIRED:

Sl.No.	COMPONENT	SPECIFICATION	QTY.
1.	D FLIP FLOP	IC 74174	1
2.	OR GATE	IC 7432	1
3.	IC TRAINER KIT	-	1
4.	PATCH CORDS	-	As required

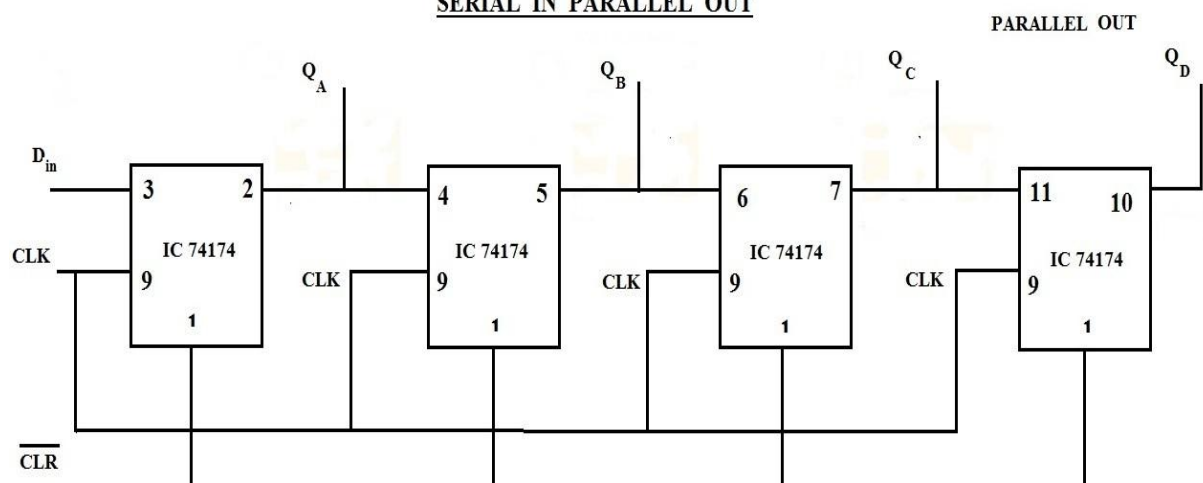
THEORY:

A register is capable of shifting its binary information in one or both directions is known as shift register. The logical configuration of shift register consist of a D-Flip flop cascaded with output of one flip flop connected to input of next flip flop. All flip flops receive common clock pulses which causes the shift in the output of the flip flop. The simplest possible shift register is one that uses only flip flop. The output of a given flip flop is connected to the input of next flip flop of the register. Each clock pulse shifts the content of register one bit position to right.

PIN DIAGRAM:**IC74174**

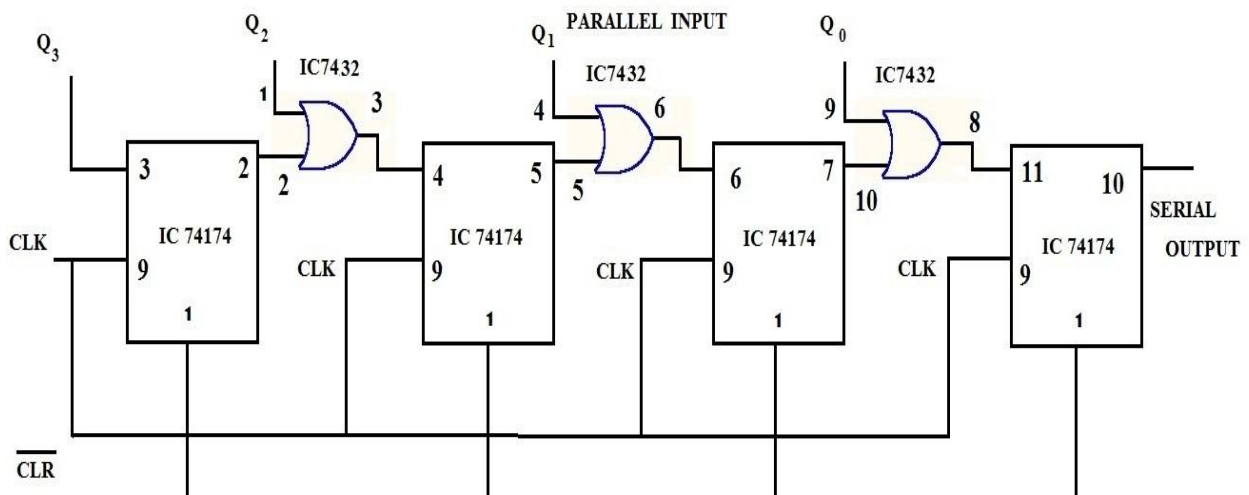
LOGIC DIAGRAM:**SERIAL IN SERIAL OUT****TRUTH TABLE:**

CLK	Serial in	Serial out
1	1	0
2	0	0
3	0	0
4	1	1
5	X	0
6	X	0
7	X	1

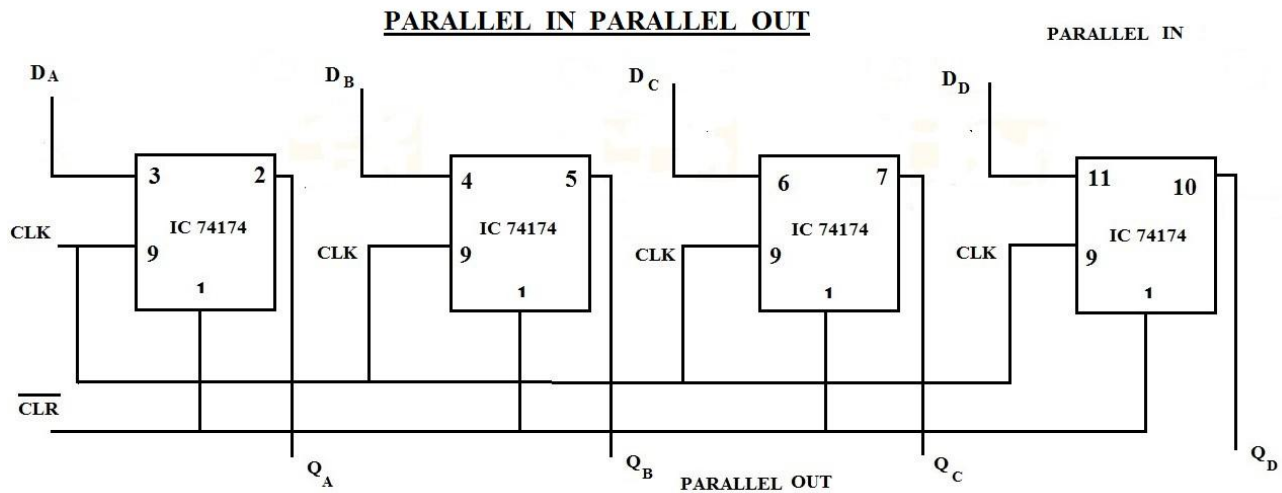
LOGIC DIAGRAM:**SERIAL IN PARALLEL OUT**

TRUTH TABLE:

CLK	DATA	OUTPUT			
		Q _A	Q _B	Q _C	Q _D
1	1	1	0	0	0
2	0	0	1	0	0
3	0	0	0	1	0
4	1	1	0	0	1

PARALLEL IN SERIAL OUT**Parallel in Serial out:****TRUTH TABLE:**

CLK	Q ₃	Q ₂	Q ₁	Q ₀	O/P
0	1	0	0	1	1
1	0	0	0	0	0
2	0	0	0	0	0
3	0	0	0	0	1

Parallel in Parallel Out:

CLK	DATA INPUT				OUTPUT			
	DA	DB	DC	DD	QA	QB	QC	QD
1	1	0	0	1	1	0	0	1
2	1	0	1	0	1	0	1	0

PROCEDURE:

1. Connections are made as per the circuit/logic diagram.
2. Make sure that the ICs are enabled by giving the suitable V_{cc} and ground connections.
3. Apply the logic inputs and observe the output by verifying the truth table given.

Viva questions:

1. What is a register?
2. Give the truth table of D flipflop.
3. What is shift register?

RESULT:

Thus the shift registers for SISO, SIPO, PISO and PIPO operations are designed and the corresponding truth tables are verified.

VHDL - OVERVIEW

1. INTRODUCTION

VHDL is Hardware description language used to describe the behavior and structure of digital systems. The acronym **VHDL** stands for **VHSIC Hardware Description Language** and **VHSIC** stands for **Very High Speed Integrated Circuit**. VHDL is a general purpose hardware description language that can be used to describe and simulate the operation of a wide variety of digital systems, ranging in complexity from a few gates to an interconnection of many complex integrated circuits. VHDL was originally developed for the military to allow a uniform method for specifying digital systems. The VHDL language has since become an IEEE standard and it is widely used in industry.

2. BASIC TERMINOLOGY:

ENTITY: A hardware abstraction of the digital system is called entity. An entity is modeled using an entity declaration and at least one architecture body. The entity declaration describes the external view of the entity. The architecture body contains the internal description of the entity.

ARCHITECTURE BODY: The internal details of an entity are specified by an architecture body using any of the following modeling styles:

- (i) Structural style of Modeling (As a set of interconnected components).
- (ii) Dataflow style of Modeling (As a set of concurrent assignment statement).
- (iii) Behavioral style of Modeling (As a set of sequential assignment statement).
- (iv) Mixed style of Modeling (As any combination of the above three).

INTERFACE PORTS: Each interface port can have one of the following modes:

1. **in** : The value of the input port can only be read within the entity model.
2. **out** : The value of the output port can only be updated within the entity model. It cannot be read
3. **inout** : The value of the bi-directional port can be read and updated within the entity model.
4. **buffer**: The value of a buffer port can be read and updated within the entity model. however it differs from the in-out mode in that it cannot have more than one source.
5. **linkage**: The value of the linkage port can be read & updated. This can be done only by another port of mode linkage. The usage of linkage is not very clear & is thus not recommended.

CONFIGURATION DECLARATION: A configuration declaration is used to create configuration for an entity. It specifies the binding of one architecture body from the many architecture bodies that may be associated with the entity. It may also specify the bindings of components used in the selected architecture body to other entities.

PACKAGE DECLARATION: It encapsulates a set of related declarations, such as type declarations, subtype declarations, &sub program declarations, which can be shared across two or more design units. A package body contains the definitions of subprograms declared in a package declaration.

3. BASIC LANGUAGE ELEMENTS

These include Data Objects that store values of a given type, Literals that represent constant values, and Operators that operate on data values.

3.1 IDENTIFIERS:

There are 2 kinds of Identifiers, Basic & Extended Identifiers.

Basic Identifiers: Composed of sequence of one or more characters. The first character must be a letter. The last character may not be an underscore.

Extended Identifiers: Sequence of characters written between 2 backslashes.

3.2 DATA OBJECTS:

Every data object belongs to one of the following classes.

1. **Constant** : It can hold a single value of given type. The value cannot be changed during simulation.
2. **Variable** : It can hold a single value of given type. Different values can be assigned to a single variable at different times.
3. **Signal** : It holds a list of values, includes current and future values.
4. **File** : It contains a sequence of values. Values can be read or written to the file.

3.3 OBJECT DECLARATION:

Object declaration is used to declare an Object, its type and its class. A value can be optionally assigned to a signal, variable, constant.

Example:

```
Constant rise_time: time := 10ns;  
Variable sum : integer range 0 to 100 := 10;  
Signal clk : bit;
```

File Declaration Syntax:

```
file file-name : file-type-name[[open mode] is string-expression];
```

3.4 DATA TYPES:

A type is a name that has associated with it a set of values and a set of operations.

The major categories are as follows

1. **Scalar types** : Values appear in sequential order.
2. **Composite types**: Composed of elements of single type(array type) or elements of different types (record type).
3. **Access types** : Provides access to objects of a given type (via pointers).
4. **File types** : Provides access to objects that contain sequence of values of a given type.

3.5 OPERATORS: The predefined operators are classified into the following six categories:

- | | |
|-------------------------|-----------------------------------|
| 1.Logical Operators. | Eg : and, or, not, nand, nor,xor. |
| 2.Relational Operators. | Eg: =, /=, <, <=, >, >=. |
| 3.Shift Operators. | Eg: sll, sla, srl, rol,ror. |
| 4.Adding Operators. | Eg: +, -, &. |

5. Multiplying Operators. Eg: *, /, mod.

6. Miscellaneous Operators. Eg: **, abs.

4. SEQUENTIAL STATEMENTS: These statements are executed serially. The following are the some examples of Sequential statements.

- (i) **if statement**: It selects a sequence of statements for execution based on the value of condition. The general form is:

```

if Boolean-expression then
    sequential statements
elsif Boolean-expression then
    sequential statements
else
    sequential statements
end if;
  
```

- (ii) **case statement**: It selects one of the branches for execution based on the values of the expression. The format of case statement is:

```

case expression is
when choices => sequential statements
when choices => sequential statements
    .
    .
when others => sequential statements
end case;
  
```

loop, null, exit, next assertion statement are some other examples of sequential statements.

- (iii) **Variable assignment statement**: It can be declared and used inside the Process statement. A variable is assigned a value using the Variable assignment statement. The syntax is given by

Variable-object := expression;

- (iv) **Signal assignment statement**: Signals are assigned values using a signal assignment statement. The syntax is given by

Signal-object <= expression [**after** delay-value];

A signal assignment statement appears within a process or outside of a process. If it occurs outside of the process, it is considered to be concurrent statement. It is considered to be sequential signal assignment statement when appears within the process.

5. CONCURRENT STATEMENTS:

These type of statements are executed in parallel manner. The following are the examples of concurrent statements.

1. **for-generation scheme** : The format is given as follows

generate-label : **for** generate-identifier **in** discrete-range **generate**

```
    block declarations
begin
    concurrent-statements
end generate generate-label;
```

2. **if-generation scheme** : The general format is given as follows

```
generate-label : if expression generate
    block declarations
begin
    concurrent-statements
end generate generate-label;
```

This scheme allows for conditional selection of concurrent statements based on the value of an expression.

- (v) **Component Instantiation** : A component instantiation statement defines a subcomponent of the entity in which it appears. It associates the signals in the entity with the ports of that subcomponent. A format is given as follows

Component-label : component –name [**port map** (association -list)];

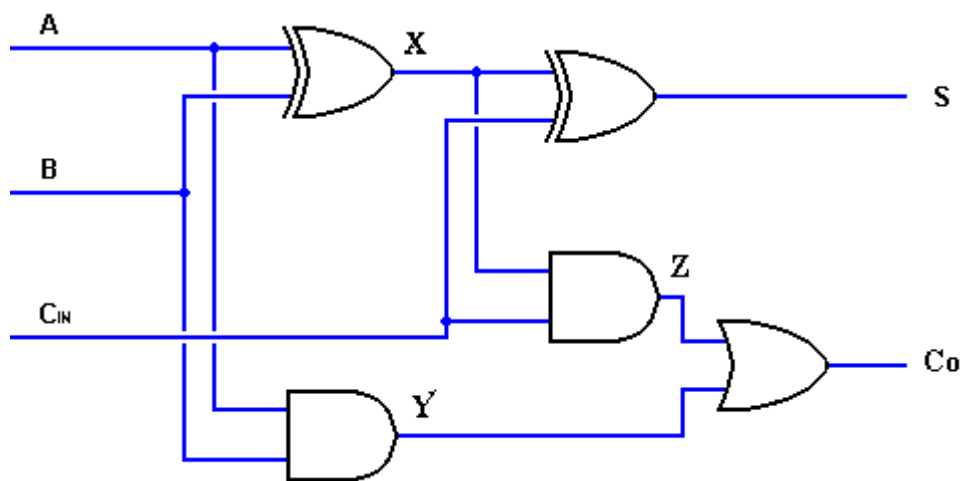
The **Block and Process** statements are also concurrent statements. The Process statement itself is a concurrent statement. The statements within Process are sequential statements

Result:

Thus the study of VHDL is done.

Ex.No:9 A SIMULATION OF COMBINATIONAL CIRCUIT using VHDL**Date:****AIM:** To verify the Functionality of a full adder and a four bit binary adder using VHDL.**THEORY:**

A full adder is capable of adding 2, 1 – bit numbers & an input carry. Four full adder circuits are needed to sum, two 4 – bit binary numbers, A&B in parallel. All the bits of A&B are applied simultaneously. The output carry from one full adder is connected to their input carry of the full adder to its left. As soon as the carries are generated, the correct sum bits emerge from the sum outputs of all full adder.

**TRUTH TABLE FOR FULL ADDER:**

Input			Output	
a	b	ci	co	s
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

FULL ADDER

```

library ieee;

use ieee.std_logic_1164.all;

entity faddr is
port (a,b,ci : in std_logic;
      s,co : out std_logic);
end faddr;

architecture struct of faddr is
signal x,y,z : std_logic;
begin
  x <= a xor b;
  y <= a and b;
  z <= x and ci;
  s <= x xor ci;
  co <= y or z;
end struct;

```

Procedure:

1. Create a project with name of the experiment.
2. Create a HDL document for the given experiment.
3. A test bench is to be created to check its functionality.

SIMULATED OUTPUT:***Full Adder:***

Now: 1000 ns	0	200	400	600
a	1			
b	0			
ci	0			
s	1			
c0	0			

RESULT:

Thus the functionality of the Full adder was verified using VHDL

Ex.No: 9B

IMPLEMENTATION OF PRIORITY ENCODER

DATE:

AIM:

To verify the Functionality of 4 to 2 Priority Encoder using VHDL

THEORY:

A priority encoder is an encoder circuit that includes the priority function. The operation of the priority encoder is such that if 2 (or) more inputs are equal to 1 at the same time, the input having the highest priority will take precedence. The truth table of a 4 input priority encoder is shown in table. The X's are don't care conditions that designate the binary value, it may be equal either to 0 or 1. Input D₃ has highest priority. D₂ has next priority level. D₀ has the lowest priority level.

TRUTH TABLE:

Input				Output		
W0	W1	W2	W3	Y0	Y1	Z
0	0	0	0	x	x	0
1	0	0	0	0	0	1
x	1	0	0	0	1	1
x	x	1	0	1	0	1
x	x	x	1	1	1	1

4 TO 2 PRIORITY ENCODER

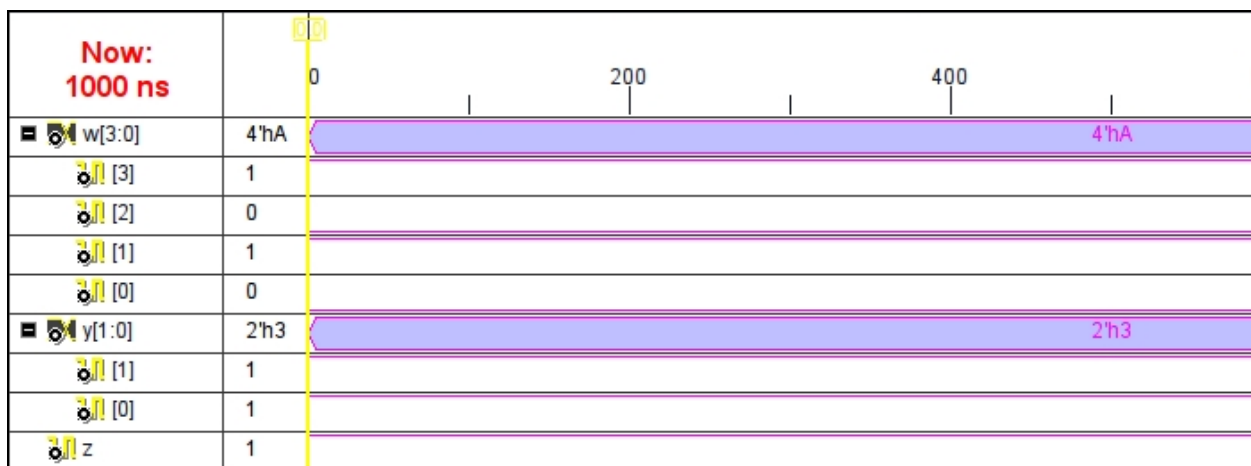
```

library ieee;
use ieee.std_logic_1164.all;
entity priority4to2 is
port(w : in std_logic_vector (3 downto 0);
      y : out std_logic_vector(1 downto 0);
      z : out std_logic);
end priority4to2;
architecture behavior of priority4to2 is
begin
y <= "11" when w(3) = '1' else
    "10" when w(2) = '1' else
    "01" when w(1) = '1' else
    "00" ;
z <= '0' when w = "0000" else '1';
end behavior;

```

Procedure:

1. Create a project with name of the experiment.
2. Create a HDL document for the given experiment.
3. A test bench is to be created to check its functionality.

SIMULATED OUTPUT :**Viva Questions:**

1. What is HDL?
2. Why HDL is essential for digital systems?
3. What is a test bench?
4. List the types of HDL programming.
5. List the difference between combinational and sequential circuits.

RESULT:

Thus the functionality of the 4 to 2 Priority Encoder was verified using VHDL

Ex.No:10 IMPLEMENTATION OF BINARY UPCOUNTER**Date:****AIM:**

To verify the Functionality of 4 – Bit Up Counter using VHDL

THEORY:

Four bit counter is capable of counting from 0 to 15. The clock inputs are connected in cascade. The enable signal is directly connected to first flipflop. Flipflops are connected through the AND gates. When enable = '0' then the inputs of all the flipflops are '0'. When the enable input = '1', it operates. The count is incremented during the rising edge of the clock pulse.

TRUTH TABLE:

Clk	Q3	Q2	Q1	Q0
0	0	0	0	0
1	0	0	0	1
2	0	0	1	0
3	0	0	1	1
4	0	1	0	0
5	0	1	0	1
6	0	1	1	0
7	0	1	1	1
8	1	0	0	0
9	1	0	0	1
10	1	0	1	0
11	1	0	1	1
12	1	1	0	0
13	1	1	0	1
14	1	1	1	0
15	1	1	1	1

Source Code:**4 –BIT BINARY UPCOUNTER**

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_unsigned.all;
entity upcount is
port(clk,rst,en : in std_logic;
```

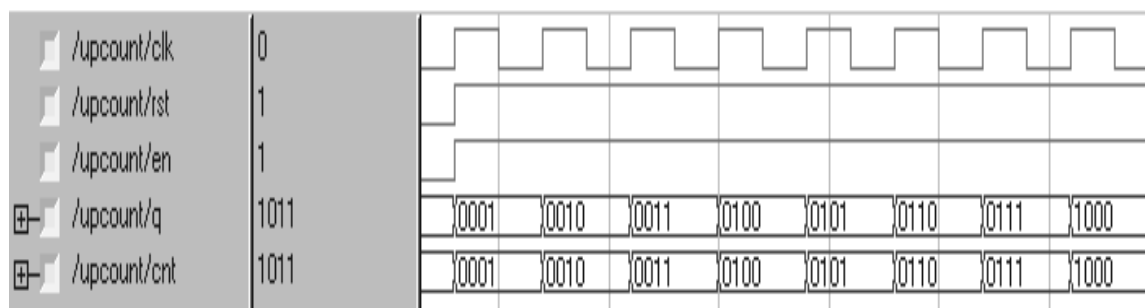
```

    q : out std_logic_vector(3 downto 0)) ;
end upcount;
architecture struct of upcount is
signal cnt : std_logic_vector(3 downto 0);
begin
process(clk,rst)
begin
if rst = '0'then
cnt <= "0000";
elsif(clk'event and clk = '1')then
    if en = '1'then
        cnt <= cnt + '1';
    else
        cnt <= cnt;
    end if;
end if;
end process;
q <= cnt;
end struct;

```

Procedure:

1. Create a project with name of the experiment.
2. Create a HDL document for the given experiment.
3. A test bench is to be created to check its functionality.

SIMULATED OUTPUT:**RESULT:**

Thus the functionality of 4 bit binary Up Counter was verified using VHDL.